Lógica para Programação

1º Ano
2º Semestre

- **Argumento**
  - É um par constituído por um conjunto de proposições, as **premissas**, e por uma única proposição, a **conclusão**; ex: ({Todos são mortais, eu sou todos}, Eu sou mortal)
  - (A, α);
  - Pode ser **válido** ou **inválido** (a conclusão é uma consequência semântica das premissas);
    - → Premissas ≡ conjunto de proposições; (podem ser V ou F)
    - → Conclusão ≡ única proposição;

- **Premissa**
  - Frase declarativa que faz uma afirmação sobre algo;

- **Símbolos Lógicos**
  - **Conjunção:**

    $\wedge \equiv e$

    $A \wedge B \equiv B \wedge A$

P	Q	P ∧ Q
V	V	V
V	F	F
F	V	F
F	F	F

  - **Disjunção:**

    $\vee \equiv ou$

    $A \vee B$

P	Q	P ∨ Q
V	V	V
V	F	V
F	V	V
F	F	F

  - **Negação:**

    $\neg P$

P	¬P
V	F
F	V

  - **Implicação:**

    $A \rightarrow B$

P	Q	P → Q
V	V	V
V	F	F
F	V	V
F	F	V

- **Lógica Proposicional** → fórmulas bem formadas
  - Símbolos de predicado são fbf, ou seja, respeitam um conjunto de regras de formação de frases da linguagem;
  - Se α e β são fbf, então $\neg\alpha$, $\alpha \wedge \beta$, $\alpha \vee \beta$, $\alpha \rightarrow \beta$, nada mais são fbf;

- **Sistema Dedutivo**
  - Alimenta-se de regras de inferência (Regra de manipulação de símbolos que especifica como formar novas fórmulas bem formadas);

1. **Regra da Premissa:**

   n → nº de linha da prova

   α → premissa

   Prem → indicação da regra

2. **Regra da Repetição:**

   n α - Repetição do α a partir da linha n;

   ⋮ ⋮ - Rep, n

3. Introdução da Conjunção:

n α
⋮ ⋮ Caminho sempre verdadeiro, a qualquer momento se α e β são verdadeiros,
m β α ∧ β é verdadeiro
⋮ ⋮
k α∧β $I_\wedge$ (n, m)

exemplo:

({P,Q}, P∧Q) ou {P,Q} ⊢ P∧Q

1	P	Prem
2	Q	Prem
3	P∧Q	$I_\wedge$(1,2)

4. Regra da Eliminação da Conjunção:

n	α ∧ β		n	α ∧ β	
⋮	⋮	ou	⋮		
m	α	$E_\wedge$, n	m	β	E_1, n

exemplo:

{P ∧ Q, R} ⊢ P∧R

1	P∧Q	Prem
2	R	Prem
3	P	E_1, 1
4	R	Rep, 2
5	P∧R	I_1(3,4)

5. Prova hipotética:

n | α hip — Considera-se α verdade

6. Regra da Reiteração:

c	β		
n	α hip		
⋮	⋮		
m	β	Reit c	Reit n

Puxa uma premissa para dentro da hipótese

7. Regra da Introdução da Implicação:

n	α	$I_\rightarrow$,(n,m)
⋮	⋮	
m	β	
m+1	α → β	

exemplo:

{P} ⊢ Q → (P∧Q)

1	P	Prem
2	Q hip	
3	P	Reit, 1
4	P∧Q	$I_\wedge$ 3,4
5	Q → (P∧Q)	$I_\rightarrow$ (2,4)

({}, P → (Q → P))

P → (Q → P) é um teorema

1	P hip	
2	Q hip	
3	P	Reit, 1
4	Q → P	$I_\rightarrow$,(2,3)
5	P → (Q → P)	$I_\rightarrow$,(1,4)

8. Regra da Eliminação da Implicação: (modus ponens)

n	α	
m	α → β	$E_\rightarrow$,(n,m)
⋮	⋮	
k	β	

exemplo:

{P → (Q → R)} ⊢ (P → Q) → (P → R)

1 $P \to (Q \to R)$ Prem
2 | $P \to Q$ hip
3 | | P hip
4 | | $P \to (Q \to R)$ Reit, 1
5 | | $Q \to R$ $E_\to$, (3,4)
6 | | $P \to Q$ Reit, 2
7 | | Q $E_\to$, (3,6)
8 | | R $E_\to$, (7,5)
9 | $P \to R$ $I_\to$, (3,8)
10 $(P \to Q) \to (P \to R)$ $I_\to$ (2,9)

9. Introdução da Negação:

n | α hip
m | B
⋮ | ⋮ } contradição
k | $\neg B$
e $\neg\alpha$

$I_\neg (n,(m,k))$

exemplo: $\{P \to Q, \neg Q\} \vdash \neg P$

1 $P \to Q$ Prem
2 $\neg Q$ Prem
3 | P hip
4 | $P \to Q$ Reit, 1
5 | Q $E_\to$ (3,4)
6 | $\neg Q$ Reit, 2
7 $\neg P$ $I_\neg (3,(5,6))$

10. Eliminação da Negação:

n $\neg\neg\alpha$
⋮ ⋮
m α $E_\neg, n$

exemplo: $\{\neg P\} \vdash P \to Q$

1 $\neg P$ Prem
2 | P hip
3 | | $\neg Q$ hip
4 | | $\neg P$ Reit, 1
5 | | P Reit, 2
6 | $\neg\neg Q$ $I_\neg (3(4,5))$
7 | Q $E_I, 6$
8 $P \to Q$ $I_\to$ (2,7)

11. Introdução da disjunção:

n | α
⋮ | ⋮
m | $\alpha \vee B$ $I_\vee, n$

n | α
⋮ | ⋮
m | $\beta \vee \alpha$ $I_\vee, n$

12. Eliminação da disjunção:

n $\alpha \vee \beta$
o | α hip
⋮ | ⋮
p | γ
R | β hip
⋮ | ⋮
s | γ
⋮ ⋮
m γ

$E_\vee, (n,(o,p),(R,S))$

exemplo: $\{P \vee Q\} \vdash Q \vee P$

1	$P \vee Q$	Prem
2	P	hip
3	$Q \vee P$	Iv, 2
4	Q	hip
5	$Q \vee P$	Iv, 4
6	$Q \vee P$	Ev (1, (2,3), (4,5))

13. Introdução da Equivalência:

$n \quad \alpha \rightarrow \beta$
$m \quad \beta \rightarrow \alpha$
$k \quad \alpha \leftrightarrow \beta \quad I\leftrightarrow (n, m)$

12. Eliminação da Equivalência:

$n \quad \alpha \leftrightarrow \beta$ ⋮ ⋮ $m \quad \alpha \rightarrow \beta \quad E\leftrightarrow, n$

ou

$n \quad \alpha \leftrightarrow \beta$ ⋮ ⋮ $m \quad \beta \rightarrow \alpha \quad E\leftrightarrow, n$

- Reduzir as Linhas de uma Prova
  - Utilizando Regras de inferência derivadas: padrão de raciocínio correspondente à aplicação de várias regras de inferência
  - Utilizando teoremas (⊢ tem premissas)
- Regra de Resolução
  - Abordagem ao sistema dedutivo baseado numa única Regra de inferência;
  - ≡ Princípio da Resolução;
  - Obriga a que as fbf estejam na forma clausal;
- Cláusula
  - Cláusula ≡ Literal ou disjunção de literais;
- Literal
  - Símbolo proposicional ou a sua negação;
- Conjunctive Normal Form
  - fbf na CNF ≡ conjunção de cláusulas;

exemplos:

CNF: $\neg A \quad A \vee \neg B \quad \neg A \wedge (B \vee C)$

Não estão na CNF: $(A \wedge B) \vee C \quad A \wedge (B \vee (D \wedge E))$

Notação:

$(P \vee \neg Q \vee \neg R) \wedge (\neg P \vee S) \wedge (Q \vee R \vee S) \equiv \{\{P, \neg Q, \neg R\}, \{\neg P, S\}, \{Q, R, S\}\}$

- Conversão para CNF

1. Eliminação da implicação:

$\alpha \rightarrow \beta \equiv \neg\alpha \vee \beta$

exemplo:

$P \rightarrow \neg(Q \vee ((R \wedge S) \rightarrow P)) \equiv \neg P \vee \neg(Q \vee ((R \wedge S) \rightarrow P)) \equiv$

$\equiv \neg P \vee \neg(Q \vee (\neg(R \wedge S) \vee P))$

2. Redução do domínio da negação:

$\neg\neg\alpha \equiv \alpha$
$\neg(\alpha \vee \beta) \equiv \neg\alpha \wedge \neg\beta$
$\neg(\alpha \wedge \beta) \equiv \neg\alpha \vee \neg\beta$

3. Obtenção da forma conjuntiva normal:

$\alpha \vee (\beta \wedge \gamma) \equiv (\alpha \vee \beta) \wedge (\alpha \vee \gamma)$

$(\neg P \vee \neg Q) \wedge (\neg P \vee R) \wedge (\neg P \vee S) \wedge (\neg P \vee \neg P) \rightarrow \{\{\neg P, \neg Q\}, \{\neg P, R\}, \{\neg P, S\}, \{\neg P\}\}$

4. Eliminação dos símbolos $\wedge$ e $\vee$ para obtenção de conjuntos;

Princípio da Resolução

- A partir de $\alpha \vee \beta$ e de $\neg\alpha \vee \gamma$ podemos concluir $\beta \vee \gamma$;
- É uma regra de inferência dedutiva: $(\{\alpha \vee \beta, \neg\alpha \vee \gamma\}, \beta \vee \gamma)$;

1	$\alpha \vee \beta$	Prem
2	$\neg\alpha \vee \gamma$	Prem
3	α	hip
4	$\neg\alpha$	hip
5	$\neg\gamma$	hip
6	α	Rep 3
7	$\neg\alpha$	Rep 4
8	$\neg\neg\gamma$	$I\neg(5,(6,7))$
9	γ	$E\neg$ 8
10	$\beta \vee \gamma$	$I\vee$,9
11	γ	hip
12	$\beta \vee \gamma$	$I\vee$ 11
13	$\beta \vee \gamma$	$E\vee(2,(9,10),(11,12))$
14	β	hip
15	$\beta \vee \gamma$	$I\vee$,14
16	$\beta \vee \gamma$	$E\vee(1,(3,13),(14,15))$

exemplo: $(\{P \to Q, Q \to R\}, P \to R)$

1	$P \to Q$	Prem
2	$Q \to R$	Prem
3	P	hip
4	$P \to Q$	Reit 1
5	Q	$E\to$,(3,4)
6	$Q \to R$	Reit 2
7	R	$E\to$,(5,6)
8	$P \to R$	$I\to$,(3,7)

$P \to Q \equiv \{\neg P, Q\}$

$Q \to R \equiv \{\neg Q, R\}$

$P \to R \equiv \{\neg P, R\}$

1 $\{\neg P, \not{Q}\}$ Prem

2 $\{\not{\neg Q}, R\}$ Prem

3 $\{\neg P, R\}$ Res (1,2)

ou

$\{\neg P, Q\}$ $\{\neg Q, R\}$

$\{\neg P, R\}$

Prova por Resolução $\Delta \vdash \alpha$

- Prova Direta
  - Passar Δ e α para a forma clausal; a partir de Δ tentar chegar a α usando resolução; (forma clausal)
- Prova por Refutação
  - Negar a conclusão: $\neg\alpha$
  - Passar Δ e $\neg\alpha$ para a forma clausal; A partir de Δ e $\neg\alpha$ chegar a uma contradição ($\{\}$):

exemplo:

forma clausal

$(\neg P \wedge \neg Q) \to \neg(P \vee Q)$ → Teorema

$\{\} \vdash (\neg P \wedge \neg Q) \to \neg(P \vee Q)$ → Negar (Refutação)

$\{\neg((\neg P \wedge \neg Q) \to \neg(P \vee Q))\}$

$\neg((\neg P \wedge \neg Q) \to \neg(P \vee Q))$

$\neg\neg((\neg P \wedge \neg Q) \wedge \neg\neg(P \vee Q)$

$\neg P \wedge \neg Q \wedge (P \vee Q)$

$\{\neg P, \neg Q, (P \vee Q)\}$

$\{\{\neg P\}, \{\neg Q\}, \{P, Q\}\}$

Prova ⇒

1	$\{\neg P\}$	Prem
2	$\{\neg Q\}$	Prem
3	$\{P, Q\}$	Prem
4	$\{Q\}$	Res, (1,3)
5	$\{\}$	Res, (2,4)

Lógica de 1ª Ordem

- Seja f_i^m um símbolo de função de aridade m ; Seja x uma variável, então:
  ↳ nº de argumentos
  - x é um termo (x, y, z, ...)
  - f_i^0 (i constantes) são termos (a, b, c, constantes)
  - Se $t_1, \ldots, t_m$ são termos, então $f_i^m(t_1, \ldots, t_m)$ é um termo (f, g, h, ...)
    ↳ símbolo de função

⚠ Nada mais é termo !!! ⚠

- exemplos (termos):
  x / Portugal / capital (Portugal) / pai (Maria) / capital (x) ...

- Seja:
  $A_1, \ldots, A_n$ termos
  P_j^m um símbolo de aridade m

↳ Então:
$P_i^m(t_1, \ldots, t_m)$ é uma fbf (atómica)

- Se α e β são fbf:
  - $\neg\alpha$, $(\alpha \wedge \beta)$, $(\alpha \vee \beta)$, $(\alpha \rightarrow \beta)$, $\exists x\,[\alpha]$, $\forall x\,[\alpha]$ são fbf.
  - ⚠ Nada mais são fbf ⚠

exemplo:
existe_fronteira (Portugal, Espanha)
símbolo de predicado

- Definição dos Predicados ⊛
  estudante (x) é verdade se x é estudante
  estuda (x, y) é verdade se x estuda y
  gosta (x, y) é verdade se x gosta de y
  diferente (x, y) é verdade se $x \neq y$
  igual (x, y) é verdade se $x = y$
  irmã_de (x, y) é verdade se y é irmã de x

- constantes: Pedro, IAED, LP

1. O Pedro é estudante → estudante (Pedro)
2. O Pedro estuda LP e IAED → estuda (Pedro, LP) ∧ estuda (Pedro, IAED)
3. O Pedro estuda LP ou IAED, mas não as duas
   ↳ (estuda (Pedro, LP) ∧ ¬estuda (Pedro, IAED)) ∨ (estuda (Pedro, IAED) ∧ (¬estuda (Pedro, LP))
4. O Pedro não estuda LP → ¬estuda (Pedro, LP)
5. Todos os estudantes são espertos → $\forall x\,[\text{estudante}(x) \rightarrow \text{esperto}(x)]$
6. Existe um estudante esperto → $\exists x\,[\text{estudante}(x) \wedge \text{esperto}(x)]$
   $\exists x\,[\text{estudante}(x) \rightarrow \text{esperto}(x)]$
   ↳ se não existe estudante é verdadeiro
7. Todo o estudante gosta de um outro estudante
   ↳ $\forall x\,[\text{estudante}(x) \rightarrow \exists y\,[(\text{estudante}(y)) \wedge \text{gosta}(x, y)]]$
8. Todo o estudante gosta de um estudante diferente de si
   ↳ $\forall x\,[\text{estudante}(x) \rightarrow \exists y\,[\text{estudante}(y) \wedge \text{gosta}(x, y) \wedge \text{diferente}(x, y)]]$

Dedução Natural em Lógica de 1ª Ordem

n $\forall x\,[\alpha(x)]$
⋮
n $\alpha(t)$ $E\forall_1$ } Regra de eliminação de 1ª ordem

exemplo:
$\{P(t),\ \forall x\,[P(x) \rightarrow \neg Q(x)]\} \vdash \neg Q(t)$

1 $P(t)$ Prem

2 $\forall x\,[P(x) \to \neg Q(x)]$ Prem

3 $P(t) \to \neg Q(t)$ $E\forall, 2$

4 $\neg Q(t)$ $E\to (1,3)$

m | x_0 |

⋮ | |

n | $\alpha(x_0)$ | não é hipótese!

n+1 $\forall x\,[\alpha(x)]$

→ Variável nova que nunca aparecem antes

$\{\forall x\,[P(x) \to Q(x)],\ \forall x\,[P(x)]\} \vdash \forall x\,[$

1 $\forall x\,[P(x) \to Q(x)]$ Prem

2 $\forall x\,[P(x)]$ Prem

3 x_0 | $\forall x\,[P(x) \to Q(x)]$ reit 1

4 | $\forall x\,[P(x)]$ reit 2

5 | $P(x_0) \to Q(x_0)$ $E\forall, 3$

6 | $P(x_0)$ $E\forall, 4$

7 | $Q(x_0)$ $E\to 5,6$

8 $\forall x\,[Q(x)]$ $I\forall\ (3,7)$

Exemplos

9. Nenhum estudante gosta do Pedro → $\neg\exists x\,[\text{estudante}(x) \wedge \text{gosta}(x, \text{Pedro})]$
   ↳ $\forall x\,[\text{estudante}(x) \to \neg\,\text{gosta}(x, \text{Pedro})]$
10. O Pedro tem pelo menos uma irmã → $\exists x\,[\text{irmã-de}(\text{Pedro}, x)]$
11. O Pedro não tem nenhuma irmã → $\neg\exists x\,[\text{irmã-de}(\text{Pedro}, x)]$
12. O Pedro tem no máximo 1 irmã → $\forall x,y\,[\text{irmã-de}(\text{Pedro}, x) \wedge \text{irmã-de}(\text{Pedro}, y) \to \text{igual}(x,y)]$
13. O Pedro tem exatamente 1 irmã
    ↳ $\exists x\,[\text{irmã-de}(\text{Pedro}, x) \wedge (\forall y\,[\text{irmã-de}(\text{Pedro}, y) \to \text{igual}(x,y)])]$
14. O Pedro tem, pelo menos, duas irmãs
    ↳ $\exists x, y\,[\text{irmã-de}(\text{Pedro}, x) \wedge \text{irmã-de}(\text{Pedro}, y) \wedge \neg(\text{igual}(x,y))]$

Dedução Natural em Lógica de 1ª Ordem

1. Eliminação do $\forall$

   n $\forall x\,[\alpha(x)]$

   ⋮

   m $\alpha(t)$ $E\forall, n$

2. Introdução do $\forall$

   n | x_0 → variável nova

   ⋮ |

   m | $\alpha(x_0)$

   m+1 $\forall x\,[\alpha(x)]$ $I\forall\ (n, m)$

3. Introdução do $\exists$

   n $\alpha(t)$

   ⋮

   m $\exists x\,[\alpha(x)]$ $I\exists, n$

exemplo:

$\{\forall x\,[P(x)]\} \vdash \exists x\,[P(x)]$

1 $\forall x\,[P(x)]$ Prem

2 $P(t)$ $E\forall, 1$

3 $\exists x\,[P(x)]$ $I\exists, 2$

4. Eliminação do ∃

n	$\exists x [\alpha(x)]$
m	x_0 | $\alpha(x_0)$ hip
⋮	⋮
k	B → fórmula que não depende de x_0
k+1	B $E_{\exists n,(m,k)}$

- exemplo:

$$\{\forall x [P(x) \to Q(x)], \exists x [P(x)]\} \vdash \exists x [Q(x)]$$

1	$\forall x [P(x) \to Q(x)]$	Prem
2	$\exists x [P(x)]$	Prem
3	x_0 | $P(x_0)$	
4	$\forall x [P(x) \to Q(x)]$	Reit 1
5	$P(x_0) \to Q(x_0)$	$E_{\forall 4}$
6	$Q(x_0)$	$E_{\to 3,5}$
7	$\exists x [Q(x)]$	$I_{\exists 3,6}$
8	$\exists x [Q(x)]$	$E_{\exists 2,(3,7)}$

- Conversão para a forma clausal

1. Eliminação da implicação $\neg\vee\wedge$
2. Redução do domínio da $\neg$ → $\neg\forall x [\alpha(x)] = \exists x [\neg\alpha(x)]$ // $\neg\exists x [\alpha(x)] = \forall x [\neg\alpha(x)]$
3. Normalização de variáveis
   ↳ Garantir que uma variável só tem um quantificador associado;
4. Eliminação do ∃
   $\exists x \alpha(x) = \alpha(a)$ → constante de skolem
   $\forall x \forall y \exists z \alpha(x,y,z) = \forall x \forall y \alpha(x,y,f(x,y))$ → função de skolem
5. Conversão para forma Prenex
   Passar os ∀ para a esquerda
6. Eliminação dos ∀
7. Distributividade
8. Obtenção de Conjuntos

- exemplo: $\forall x [P(x) \to (\forall y [P(y) \to P(f(x,y))] \wedge \neg \forall y [Q(x,y) \to P(y)])]$

(1) $\forall x [P(x) \to (\forall y [\neg P(y) \vee P(f(x,y))] \wedge \neg \forall y [\neg Q(x,y) \vee P(y)])]$

$\forall x [\neg P(x) \vee (\forall y [\neg P(y) \vee P(f(x,y))] \wedge \neg \forall y [\neg Q(x,y) \vee P(y)])]$

(2) $\forall x [\neg P(x) \vee (\forall y [\neg P(y) \vee P(f(x,y))] \wedge \exists y [\neg\neg Q(x,y) \wedge \neg P(y)])]$

(3) $\forall x [\neg P(x) \vee (\forall y [\neg P(y) \vee P(f(x,y))] \wedge \exists y [Q(x,y) \wedge \neg P(y)])]$

$\forall x [\neg P(x) \vee (\forall y [\neg P(y) \vee P(f(x,y))] \wedge \exists z [Q(x,z) \wedge \neg P(z)])]$

(4) $\forall x [\neg P(x) \vee (\forall y [\neg P(y) \vee P(f(x,y))] \wedge [Q(x,g(x)) \wedge \neg P(g(x))])]$

(5.6) $\neg P(x) \vee [\neg P(y) \vee P(f(x,y))] \wedge [Q(x,g(x)) \wedge \neg P(g(x))]$

(7) $(\neg P(x) \vee \neg P(y) \vee P(f(x,y))) \wedge (\neg P(x) \vee Q(x,g(x))) \wedge (\neg P(x) \vee \neg P(g(x)))$

- Resolução

- $\{P, A\}, \{\neg P, B\}$ → $\{A, B\}$

- $\{P(x), R(x)\}$ $\{\neg P(c), Q(a)\}$ → $\{c/x\}$ → substitui o x por c
  ↳ ultrapassar generalização
  → $\{R(c), Q(a)\}$

- Substituição
  - $S_1 = \{t_1/x_1, \ldots, t_n/x_n\}$
  - $S_2 = \{u_1/y_1, \ldots, u_n/y_n\}$
- Composição de Substituição ($S_1 \circ S_2$)
  - Tal que para uma fbf α: $\alpha \cdot (S_1 \circ S_2)$ resulta em:
    1. Aplica-se S_2 aos termos de S_1;
    2. Adicionam-se os elementos de S_2 que dizem respeito a variáveis que não ocorrem em S_1.
    3. Removem-se $(t_i \circ S_2)/x_i$, tais que $t_i \circ S_2 = x_i$ (não podemos ter x/x)
  - exemplo:

$S_1 = \{f(y)/x, z/y\}$ (t_1, x_1, t_2, x_2) $\quad S_2 = \{a/x, b/y, y/z\}$

$S_1 \circ S_2$

1. $\{f(y)^{t_1} \cdot \{a/x, b/y, y/z\}^{S_2}/x, (z^{t_1}:\{a/x, b/y, y/z\}^{S_2})/y, a/x\ b/y, y/z\}$
2. $\{f(b)/x, y/y, a/x, b/y, y/z\}$

   x e y já ocorriam em S_1
3. $S_1 \circ S_2 = \{f(b)/x, y/z\}$

- Um conjunto de fbf atómicas $\{\alpha_1, \ldots, \alpha_n\}$ diz-se unificável se existir uma substituição que torna idênticas todas as suas fbf $\alpha_1 \cdot S = \ldots = \alpha_n \cdot S$
  ↳ unificador do conjunto

exemplo: $\{P(a, y, z), P(x, b, z)\}$

unificador:

$S_1 = \{a/x, b/y, c/z\}$
$S_2 = \{a/x, b/y, d/z\}$
$\vdots$

} possibilidades

- Unificador Mais Geral
  - Dado um conjunto de fbf atómicas $\{\alpha_1, \ldots, \alpha_n\}$. O unificador mais geral (UMG) é o unificador s tal que se S_1 é um unificador de $\{\alpha_1, \ldots, \alpha_n\}$ então existe S_2 tal que $S_1 = S_1 \circ S_2$, o UMG é o único
  - exemplo: $\Delta = \{P(a, x, f(y)), P(u, v, w), P(a, R, f(c))\}$

x, y, u, v, w, R são variáveis
a, c são constantes

	Conjunto de Discordância	Substituições
$\{P(a, x, f(y)), P(u, v, w), P(a, R, f(c))\}$	$\{u, a\}$	$\{a/u\}$ → única hipótese
$\{P(a, x, f(y)), P(a, v, w), P(a, R, f(c))\}$	$\{u, v, R\}$	$\{v/x, v/R\}$ → escolha
$\{P(a, v, f(y)), P(u, v, w), P(a, R, f(c))\}$	$\{f(c), w, f(y)\}$	$\{c/y, f(c)/w\}$
$\{P(a, v, f(c))\}$ ↳ Como o cardinal é 1, acabou.		UMG ↓ $\{a/u\} \circ \{v/x, v/R\} \circ \{c/y, f(c)/w\} = \{a/u, v/x, u/R, c/y, f(c)/w\}$

- exemplos:

1. Os antepassados dos nossos ascendentes diretos são nossos antepassados;
2. Os nossos ascendentes diretos são nossos antepassados;
3. O Pedro é ascendente direto da Ana;
4. O Rui é um ascendente direto do Pedro;

a → O Rui é antepassado da Ana?

AD(x,y): x é ascendente direto de y

ant(x,y): x é antepassado de y

1. $\forall x,y,z\ [ant(x,y) \wedge AD(y,z) \rightarrow ant(x,z)]$
2. $\forall x,y\ [AD(x,y) \rightarrow ant(x,y)]$
3. AD(Pedro, Ana)
4. AD(Rui, Pedro)

forma clausal →

1 {¬and(x,y), ¬AD(y,z, ant(x,z)}
2 {¬AD(x,y), ant(x,y)}
3 {AD(Pedro, Ana)}
4 {AD(Rui, Pedro)}

a)

1 {¬AD(x,y), ant(x,y)} prem
2 {¬ant(x,y), ¬AD(y,z), ant(x,y)} prem
3 {AD(Pedro, Ana)} prem
4 {AD(Rui, Pedro)} prem
5 {¬ant(Rui, Ana)} prem
6 {ant(Rui, Pedro)} Res(1,4) {Rui/x, Pedro/y}
7 {¬AD(Pedro, z), ant(Rui, z)} Res(2,6) {Rui/x, Pedro/y}
8 {ant(Rui, Ana)} Res(3,7) {Ana/z}
9 {} Res(5,8)

→ conclusão: {ant(Rui, Ana)} ↓ negar {¬ant(Rui, Ana)}

Cláusula de Horn

- Cláusula que contém no máximo um literal positivo;
- exemplo:

{c, ¬P1, ¬P2}
{c}
{¬P1, ¬P2}
} cláusulas de Horn

{c, P1} → não é

- Ao literal positivo, se existir, chamamos cabeça da cláusula;
- Aos literais negativos, se existirem, chamamos corpo da cláusula;
- Cláusula vazia □;
- Representação alternativa cabeça ← corpo;
- exemplo:

{c, ¬P1, ¬P2} c ← P1, P2 Regra
{c} c ← afirmação
{¬P1, ¬P2} ← P1, P2 objetivo

Tipos de Cláusulas de Horn

- Regras ou implicação (cabeça e corpo não vazios);
- Afirmação ou factos: corpo vazio;
- Objetivo: cabeça vazia;
- Cláusula vazia: corpo e cabeças vazios;

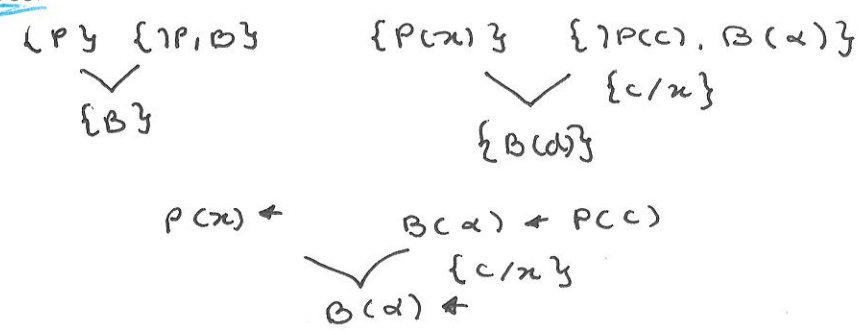
Resolução com Cláusulas de Horn

- $\alpha \leftarrow \gamma_1, \gamma_n$
- $\delta \leftarrow B_1, \ldots, B_i, \ldots, B_i, B_{i+1}, B_n$

$(\delta \leftarrow B_1, \ldots, B_{i-1}, \gamma_1, \ldots, \gamma_n, B_{i+1}, \ldots, B_n), \sigma$ ↳ UMG para α

* se δ não existir, tem um objetivo (prolog) *

Nota:



- exemplos:

Forma Clausal	Cláusulas de Horn
$\{\neg AD(x,y), Ant(x,y)\}$	$Ant(x,y) \leftarrow \neg AD(x,y)$
$\{\neg Ant(x,y), \neg AD(y,z), Ant(x,z)\}$	$Ant(x,z) \leftarrow \neg Ant(x,y), \neg AD(y,z)$
$\{AD(Pedro, Ana)\}$	$AD(Pedro, Ana) \leftarrow$
$\{AD(Rui, Pedro)\}$	$AD(Rui, Pedro) \leftarrow$
$\{\neg Ant(Rui, Ana)\}$	$\leftarrow Ant(Rui, Ana)$

1 $Ant(x,y) \leftarrow AD(x,y)$ prem
2 $Ant(x,z) \leftarrow Ant(x,y), AD(y,z)$ prem
3 $AD(Pedro, Ana) \leftarrow$ Prem
4 $AD(Rui, Pedro) \leftarrow$ Prem
5 $\leftarrow Ant(Rui, Ana)$ Prem
6 $Ant(Rui, Pedro) \leftarrow$ Res(1,4), {Rui/x, Pedro/y}
7 $Ant(Rui, z) \leftarrow AD(Pedro, z)$ Res(2,6) {Rui/x, Pedro/y}
8 $Ant(Rui, Ana) \leftarrow$ Res(3,7) {Ana/z}
9 □ Res(5,8) {}

○ Programa

- Conjunto finito de cláusulas;
- Determinadas ↔ Regras / Factos
- Seja Δ um programa e α um objetivo. A resposta de Δ a α é uma substituição (pode ter razia) para as variáveis de α.
  ↳ Δ diz-se uma resposta correta se $\Delta \cup \{\neg\alpha . \sigma\}$ for contraditório;

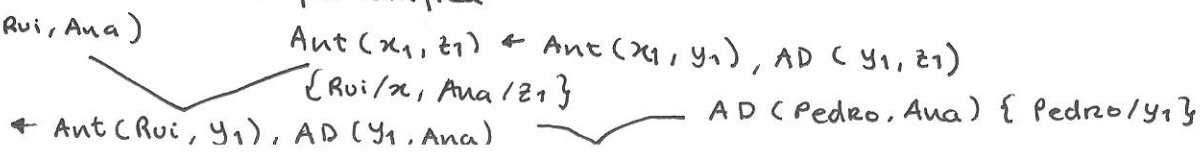
○ Resolução SLD

1. Regra de seleção (dos literais do objetivo)
2. Regra de procura (de cláusulas)
3. Estratégia de resolução linear (cláusula inicial + sucessores)

- exemplo:

Programa Anterior Δ ⑧

Objetivo ← Ant(Rui, Ana)
Função de seleção: $S(\leftarrow \alpha_1, \ldots, \alpha_n) = \alpha_n$
Regra de Procura: a última que unifica



Resolução Linear



○ Prova SLD

- Sequência de objetivos $[\gamma_0, \ldots \gamma_n]$
- Se $\gamma_n = \square$ diz-se que a prova SLD é uma Refutação SLD, supondo que foram feitas as substituições $s_0, \ldots, s_n$, a Resposta de Δ a α é

$$(s_0 \circ s_1 \circ \ldots \circ s_n)\Big|_{V(\alpha)}$$

$V(\alpha)$ → variáveis de α

› Árvore SLD

- Δ programa
- α objetivo
- S função de seleção

Explorar todas as hipóteses

- Ramo cuja folha tem $\square$ diz-se bem sucedido;
- Ramo que não tem $\square$ diz-se falhado;
- Os restantes são infinitos;

exemplo da solução:

1. $(\{x/x_1, b/z_1\} \circ \{a/x_1, b/x_1\} \circ \{b/x_3\})\Big|_{V(x)} = \{a/x\}$
2. $(\{b/x, b/x_4\})\Big|_{\{x\}} = \{b/x\}$

○ Prolog

- Programa em prolog:



○ Componentes Básicos

- Constantes : a, 'a', 3
- variáveis : X, Y, _X → variável especial
- Termos compostos ≡ função mais argumento;
- Literais → predicados + argumentos
- Programa { Factos → estudante(maria)
  Regras → da(x,y) :- oi(X,Y)
- Objetivo

Comentários
% /* */

○ Semântica do Prolog

- Programa + objetivo + Refutação SLD

* função de seleção: escolhe o 1º literal do objetivo;
Regra de prova: escolhe a 1ª cláusula unificável com o literal do objeto em estudo;

- Programa

C_1 :- ...

⋮

C_j :- corpo [...]

⋮

C_N :- ..

- Objetivo

? - $O_1, O_2, \ldots, O_n$

assumindo que O_1 é unificável com C_i

? - corpo, $O_2, \ldots, O_n$

- exemplo:

c1 amigo(harry, ron).
c2 amigo(lupin, harry).
c3 ligado(X, Y) :- amigo(X, Y).
c4 ligado(X, Z) :- ligado(X, Y), amigo(Y, Z).

? - ligado(lupin, ron)

ligado(X, Y) :- amigo(X1, Y1)
{lupin/X1, ron/Y1}

? amigo(lupin, ron)

X

ligado(X1, Z1) :- ligado(X1, Y1), amigo(Y1, Z1)
{lupin/X1, ron/Z1}

? - ligado(lupin, Y1), amigo(Y1, ron)

ligado(X2, Y2) :- amigo(X2, Y2)
{lupin/X2, Y1/Y2}

? amigo(lupin, Y1)

amigo(lupin, harry)
{Y1/harry}

? amigo(harry, ron)

amigo(harry, ron).

□
True

- Unificação de Termos

- = (predicado da unificação)
- <t1> = <t2> tem sucesso se <t1> e <t2> podem ser unificados.
- \= (negação do predicado da unificação <t1> \= <t2> tem sucesso se <t1> e <t2> não podem ser unificados.

Nota: = faz a unificação, mas não a avaliação numérica. is avalia numericamente a expressão do lado direita e unifica com o lado esquerdo.

- Comparação de Termos

- == (predicado identidade)

<t1> == <t2> tem sucesso se <t1> e <t2> são idênticos;

\== <t1> \== <t2> tem sucesso se <t1> e <t2> não são idênticos.

a == a	x == y	True
a == 'a'	x = a, x == a	
a == b	x == a, x = a	False

- Listas

[elem 1, ..., elem n]

[ ] → lista vazia

[(1,2), (3,4)]

[[0, 1, _, 1], [1, _, _, _], [0, 1, 0, 1]]

exemplo:

?_ [HEAD | TAIL] = [dp, th, fd]

1º elemento
[HEAD | TAIL]
↓
Lista s/ o primeiro elemento

HEAD = dp,
TAIL = [th, fd]

- [dp, th, fd] = [dp|T]
  T = [th, fd]

[x] - lista com um único elemento (que pode ser uma lista)
[H|T] não unifica com []

Soma

soma([H|T], N) :- soma(T, M), N is M + H.

soma([1, 4, 2], N)

soma([4, 2], m), N is m + 1

Junta

junta(L_1, L_2, L_3) é verdade se L_3 for o resultado de juntar as listas 1 e 2.

junta([P|R], L_1, [P|L_2]) :- junta(R, L_1, L_2).

Recursão

① Engordar → método iterativo

comp2(L,C) :- comp(L, 0, C).
↳ Inicializado ②

comp2([], C, C). ④ Transferir

comp2([_|cauda], A, C) :- A_1 is A + 1, comp2(cauda, A_1, C).

findall

③ Implementação

$$factorial\ n! = \begin{cases} 1 \text{ se } n = 1 \\ n(n-1) \text{ se } n > 1 \end{cases}$$

Recursiva

em prolog:

factorial(1, 1)

factorial(N, F) :- N > 1, N_1 is N - 1, factorial(N_1, F_1), F is N * f_1.

Negação / Corte

- falhanço forçado: fail

Negação: \+(P) :- P, !, fail
\+(P).
ou not.

Corte

R :- p_1, ..., p_m, !, q_1, ..., q_m.

Quando o corte é atingido todas as alternativas possíveis e ainda não exploradas para R p_1, ..., p_m, não são consideradas independentemente do que acontecer a q_1, ... q_m.

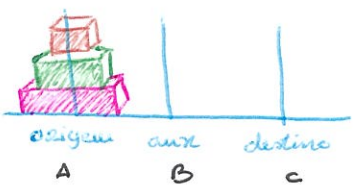
Funções Importantes Prolog

findall(T, G, Bag): Bag é a lista com as instâncias de T que verificam G.

member(x, [List]): True se x é um elemento de List

Torres de Hanoi

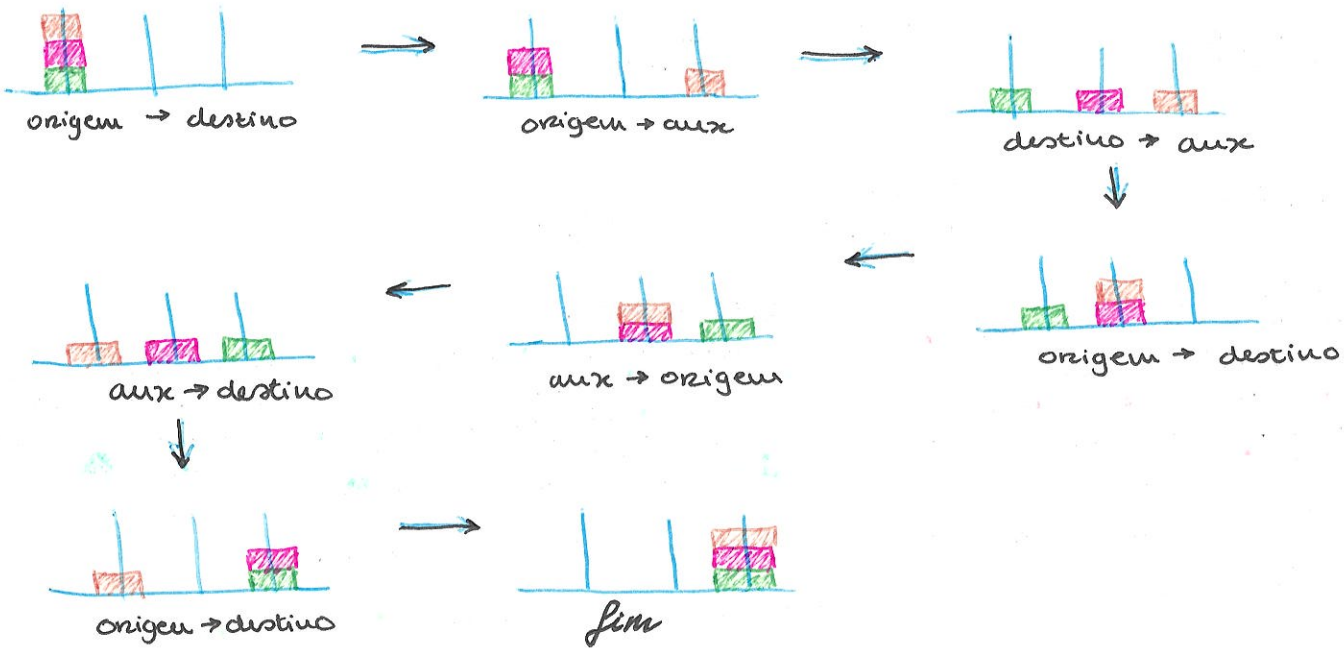
①



Intuição: mover N discos de A para C é o mesmo que:
a) mover os N-1 primeiros discos de A para B.
b) o último mover para C.
c) mover os N-1 discos de B para C.

hanoi :- write('Num de discos:'), read(N), number(N), move(N, origem, destino, aux).
move(1, origem, destino, _) :- write(origem), write('->'), write(destino), nl.

```
move(N, origem, destino, aux) :- N>1, M is N-1,
                                 move(M, Origem, Aux, Destino),
                                 move(1, Origem, Destino, _),
                                 move(M, Aux, Destino, Origem).
```



- Função Projeto

maplist(G, L): Verdade se G pode ser aplicado a todos os elementos L;
include(G, L1, L2): filtra elementos que verificam G de L1 e coloca-os em L2.
exclude(G, L1, L2): filtra elementos que não verificam G e coloca-os em L2.

- Estruturas

exemplo:

```
data(ano, mes, dia).
ano-de(data(A,_,_), A).
mes-de(data(_, M, _), M).
dia-de(data(_,_,D), D).
muda_ano(A, data(_,M,D), data(A, M, D)).
```

Homoiconicidade: permite que um programa se modifique a si próprio;

Predicado estático: não podem ser alterados (default);

Predicados dinâmicos: podem ser alterados;

Declaram:
`:- dynamic pred/aridade`
exemplo:
`:- dynamic mais alto/2`
`:- dynamic mais baixo/2`

```
?- assert a (maisAlto(h,t)).
?- assert a (maisAlto(h,cA)).
?- assert z (maisBaixo(x, y) :- maisAlto(y, x)).
```

Nota:
assert(c) → adiciona c no início
assertz(c) → " " c no fim

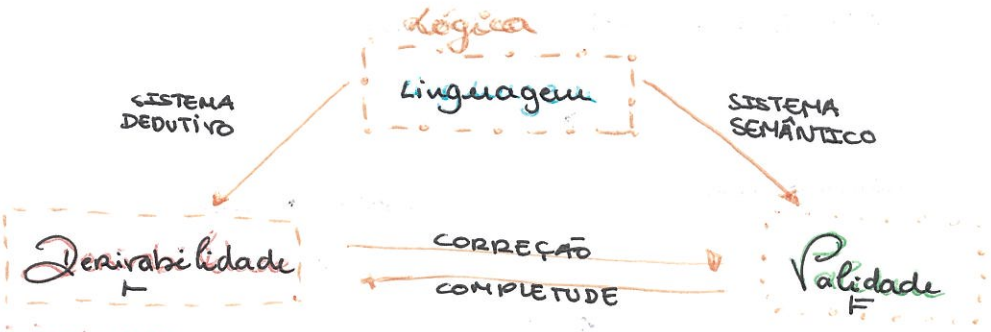
Retract(c): remove a 1ª cláusula que unifica c.

Retractall(pred): remove todas as cláusulas cuja cabeça é pred.

- Componentes de uma Lógica

Linguagem: A linguagem de uma lógica é definida com base num conjunto de regras de formação, que especificam frases legais da lógica (fbf).
Argumentos: (Δ, α), $\Delta \in \alpha$ (conjunto ou / de fbf)

Um argumento é válido se e só se é impossível ter todas as premissas verdadeiras e a conclusão falsa.



SISTEMA DEDUTIVO:
→ Manipulação de símbolos; provas (sequências de fbf); com base nas premissas tentamos chegar à conclusão. Se $\Delta \vdash \alpha$, o argumento é demonstrável.

SISTEMA SEMÂNTICO:
→ Atribuímos significado (valores lógicos) aos símbolos. Através destas atribuições determinamos a validade/invalidade de um argumento. Se $\Delta \models \alpha$, α é consequência lógica de Δ, ou Δ implica logicamente α.

- Uma lógica diz-se completa, se todo o argumento válido é demonstrável.
- Uma lógica diz-se correta, se todo o argumento demonstrável é válido.

O sistema semântico: Lógica Proposicional

DEF Função de valoração

$$v: \mathcal{P} \to \{V, F\}$$

$\mathcal{P}$: símbolos de proposição

DEF Interpretação

$I: \mathcal{L} \to \{V, F\}$

I define-se recursivamente do seguinte modo:

- $I(\alpha) = v(\alpha)$ se $\alpha \in P$
- $I(\neg\alpha) = V$ sse $I(\alpha) = F$
- $I(\alpha \wedge \beta) = V$ sse $I(\alpha) = V$ e $I(\beta) = V$
- $I(\alpha \vee \beta) = V$ sse $I(\alpha) = V$ ou $I(\beta) = V$
- $I(\alpha \to \beta) = V$ sse $I(\alpha) = F$ ou $I(\beta) = V$

exemplo: $\alpha = (P \wedge Q) \to R$

P	Q	R	P∧Q	(P∧Q) → R
V	V	V	V	V
V	V	F	V	F
V	F	V	F	V
V	F	F	F	V
F	V	V	F	V
F	V	F	F	V
F	F	V	F	V
F	F	F	F	

$I(\alpha) = F$

DEF satisfação

Diz-se que I satisfaz α sse $I(\alpha) = V$

- α satisfazível se e só se existe pelo menos uma interpretação I tal que $I(\alpha) = V$.
- α é falsificável se e só se existe pelo menos uma interpretação I tal que $I(\alpha) = F$.
- α é uma tautologia se e só se $I(\alpha) = V$ para todas as interpretações.
- α é uma contradição se e só se $I(\alpha) = F$ para todas as interpretações.

Conjunto Satisfazível: existe uma interpretação que satisfaz todas as suas fórmulas;

Conjunto Contraditório: se e só se não existe nenhuma interpretação que satisfaz todas as suas fórmulas; ex: $\{P, \neg P\}$

Modelo de um Conjunto de fórmulas: dado um conjunto de fbf Δ, uma interpretação que satisfaz todas as fbf de Δ diz-se um modelo de Δ.

$M(\Delta, \alpha)$ é válido se e só se todos os modelos de Δ são modelos de α.

Teorema da Refutação

Δ conjunto de fbf
α fbf

$\Delta \models \alpha$ se e só se $\Delta \cup \{\neg\alpha\}$ não é satisfazível.

↓
validade

Árvores de Decisão

Árvores binárias em que:

1. nós são símbolos proposicionais
2. folhas são V ou F
3. Ramos —— verdadeiro
   - - - falso

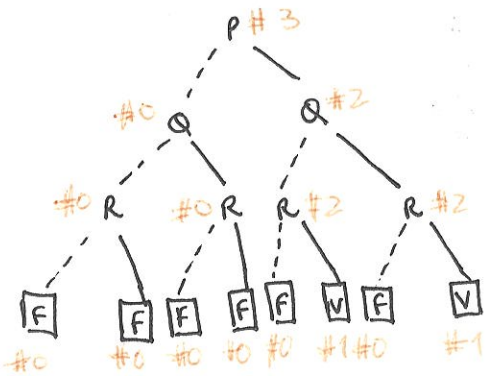
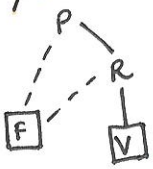


tabela associativa:



filhos iguais → mesma marca

compacta:



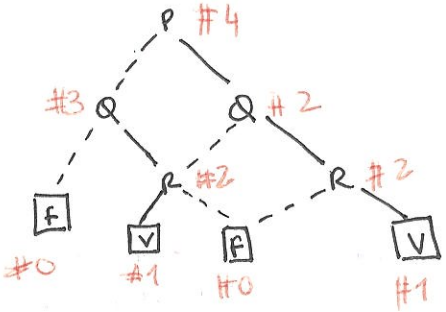
ideia: transformar árvore em grafo acíclico e dirigido.

OBDDS Grafo dirigido acíclico e rotulado que satisfaz a relação de ordem total para os seus símbolos:

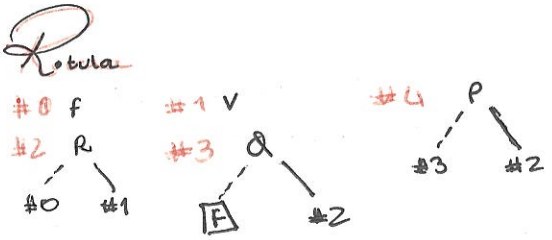
$\{P_1, \ldots, P_n\}$ símbolos de proposição
$<$ Relação de ordem
total → permite comparar todos os elementos do conjunto.

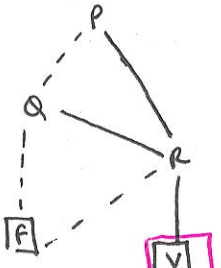
exemplo:

Algoritmos Rotula & Compacta



Rotula





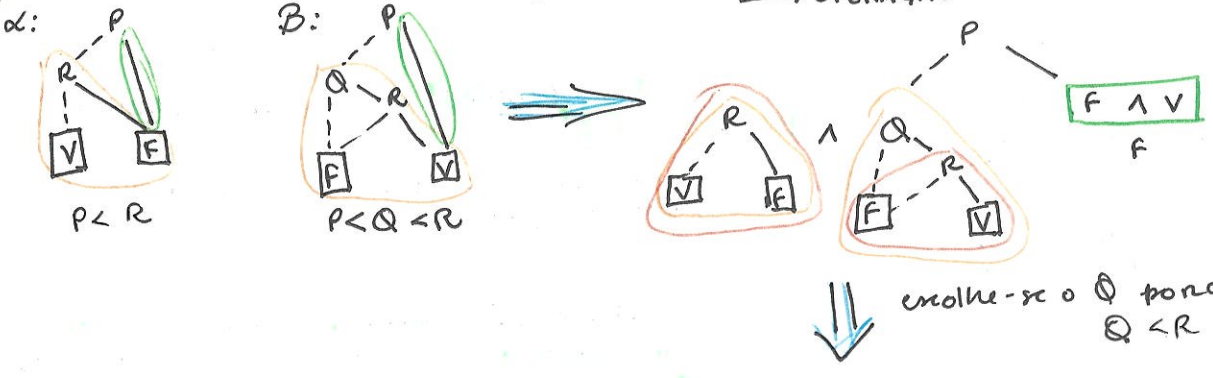
Compacta

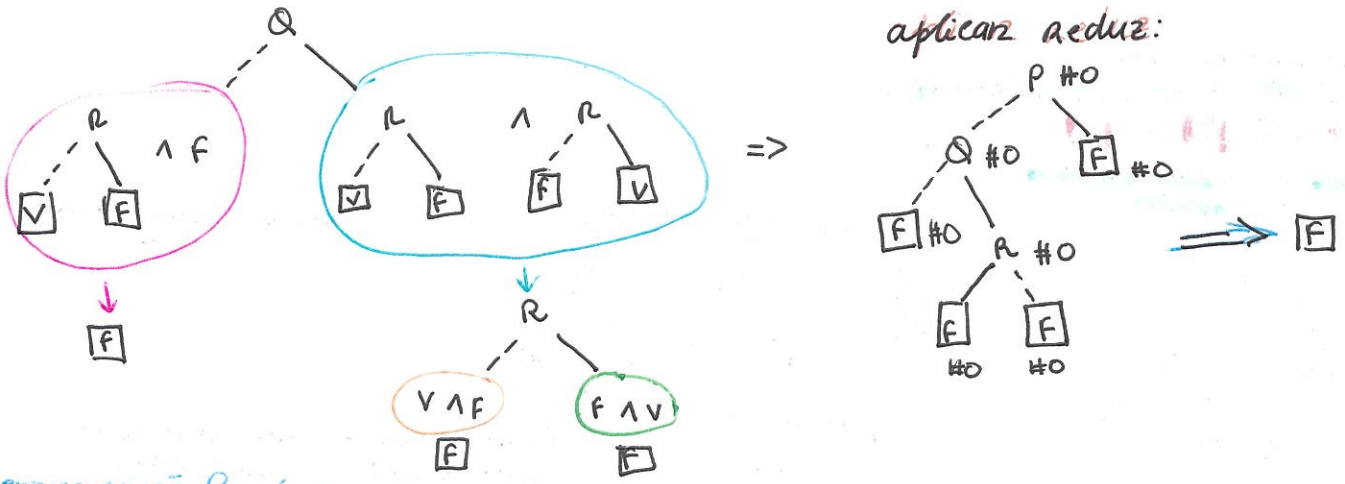
Modelos:

$I(R) = V, I(P) = V$ $\left\langle \begin{matrix} I(Q) = V \\ I(Q) = F \end{matrix} \right.$

$I(R) = V, I(Q) = V, I(P) = F$

aplica:





Representação Canónica de OBDD's

Possibilita a realização de vários testes:

- ausência de símbolos redundantes;
- teste de validade;
- teste de satisfação;
- teste de implicação ($\alpha \to \beta$ sse OBDD resultado de $\alpha \wedge \neg\beta$ é F).

Algoritmo de SAT (Satisfiability)

Permite apenas saber se uma fbf é satisfazível ou não (se sim, identificar uma testemunha (modelo)).

OBDD's Permitem identificar todos os modelos de uma fbf (interpretações que satisfazem a fbf);

Algoritmos:

1. Propagação de marcas (eficiente, mas não completo).
2. DP (menos eficiente, mas completo).

Algoritmo de Propagação de Marcas

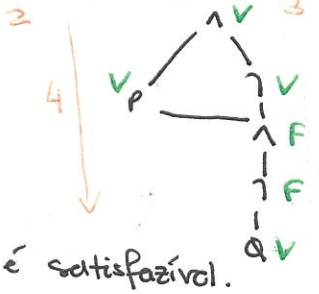
1. Obter uma fbf que só contém conjunções e negações
   $\alpha \vee \beta \equiv \neg(\neg\alpha \wedge \neg\beta)$
   $\alpha \to \beta \equiv \neg(\alpha \wedge \neg\beta)$
   $\neg\neg\alpha \equiv \alpha$
2. Construir um grafo dirigido acíclico (DAG)
3. Marcar o topo do DAG com um V
4. Propagar marcas tendo em conta as regras para a $\wedge$ e $\neg$.

exemplo:

1 $\neg(P \to \neg(\neg P \vee Q))$

$\Rightarrow \neg(\neg P \vee (\neg\neg P \wedge \neg Q)) \equiv \neg\neg P \wedge \neg(P \wedge \neg Q) \equiv$

$\equiv P \wedge \neg(P \wedge \neg Q)$

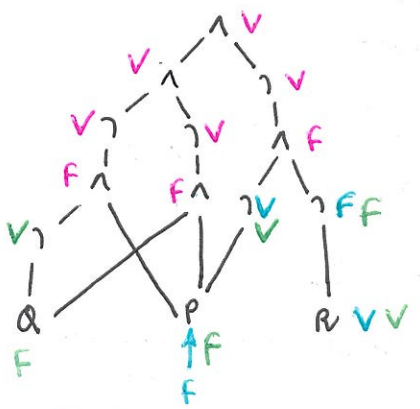


Testemunha (modelo): $I(P) = V$, $I(Q) = V$. A fbf é satisfazível.

Teste de Nós

exemplo: $(P \to Q) \wedge (P \to \neg Q) \wedge (P \vee R)$

$(\neg(P \wedge \neg Q) \wedge \neg(P \wedge Q) \wedge \neg(\neg P \wedge \neg R)$



Nós teste - Cavalos — Encadeados

marcas permanentes

teste de nós:

marcas temporárias que podem passar a permanentes

(só pode ser chamado novamente se tudo tiver carácter permanente)

teste de um nó

hip. 1: atribuição de marcas consistentes a todos os nós.

hip. 2: propagação do valor atribuído (V/F) dá origem a uma contradição, logo é atribuído o valor simétrico

(F/V) a esse nó com caracter permanente;

hip. 3: nenhuma das hipóteses anteriores... mas: marcas temporárias comuns aos dois valores passam a permanentes.

exemplo: $I(P)=V \xrightarrow{\text{Prop.}}$): V; $I(P)=F \xrightarrow{\text{Prop}}$): V — nó: V — passa a permanente

Algoritmo DP

ideia: seja Δ uma fbf e P um símbolo de proposição, Δ satisfazível se o conjunto obtido por eliminação de P for satisfazível.

$\exists P(\Delta)$ → eliminação de P em Δ - como se calcula?

Todos os Resolventes - P são adicionados

Todas as cláusulas que contêm P são Retiradas

exemplo:

$\Delta = \{\{P,Q\},\{\neg P,Q\},\{\neg P,S\},\{\neg Q,R\}\}$

$\exists P(\Delta) = \{\{Q\},\{Q,S\},\{\neg Q,R\}\}$

Aux:

$\{P,Q\}\ \{\neg P,Q\} \rightarrow \{Q\}$

$\{P,Q\}\ \{\neg P,S\} \rightarrow \{Q,S\}$

teorema: Seja Δ uma fbf na forma clausal e P_i um símbolo de proposição. Então Δ é satisfazível se e só se $\exists P_i(\Delta)$ é satisfazível.

algoritmo DP:

1. Eliminar sucessivamente os símbolos de proposição de Δ.
2. Condição de paragem:
   2.1. Geração de cláusula vazia ($\{\} \in \Delta$): fbf inicial não é satisfazível.
   2.2. Obtenção de um conjunto vazio ($\{\} = \Delta$): fbf inicial é satisfazível.

exemplo 1

$\Delta = \{\{P,Q\},\{\neg P,Q\},\{\neg Q,R\},\{\neg R\}\}$

$\exists P(\Delta) = \{\{Q\},\{\neg Q,R\},\{\neg R\}\}$

$\exists Q(\exists P(\Delta)) = \{\{R\},\{\neg R\}\}$

$\exists R(\exists Q(\exists P(\Delta))) = \{\} \rightarrow \Delta$ é satisfazível

exemplo 2

$\Delta = \{\{P,Q\},\{\neg P,Q\},\{\neg Q,R\}\}$

$\exists P(\Delta) = \{\{Q\},\{\neg Q,R\}\}$

$\exists Q(\exists P(\Delta)) = \{\{R\}\}$

$\exists R\{\exists Q(\exists P(\Delta)) = \{\} \rightarrow$ satisfazível

- implementação usando baldes:
  - Ordem nos símbolos de proposição;
  - um balde para cada símbolo;
  - cada cláusula vai parar ao 1º balde que mencione 1 dos seus símbolos.
  - Processar cada balde:
    - em cada balde bp: gerar os resolventes de P_i e colocar no balde respetivo.
    - o algoritmo termina com a geração de $\{\}$ (não satisfazível) ou após não se conseguir fazer nada (satisfazível)

exemplo: $\Delta = \{\{P,Q,\neg R\},\{\neg P,S,T,R\},\{\neg Q,\neg R\},\{S\},\{\neg P,Q,S\}\}$ ordem: $P<Q<R<S<T$

b_P: $\{P,Q,\neg R\}$ ① $\{\neg P,S,T,R\}$ ② $\{\neg P,Q,S\}$ ⑤ → C. aux: $\{P,Q,\neg R\}\ \{\neg P,S,T,R\}$ ⑪ $\{P,Q,\neg R\}\ \{\neg P,Q,S\}$

b_Q: $\{Q,\neg R\}$, $\{Q,\neg R,S\}$ → C Aux: $\{Q,\neg R\}\{Q,\neg R,S\}$ $\{Q,\neg R,S,T,R\}$ sai pois é um teorema; $\{Q,\neg R,S\}$

b_R: $\{\neg R,S\}$ FIM - ($\neg R$'s)

b_S: $\{S\}$

b_T:

testemunha:
I(R) = V (opcional) I(T) = V (opcional)
I(Q) = F (obrigat) I(S) = V (obrigatório)
I(P) = V (obrigatório)

Lógica de 1ª Ordem (LPO)

A(a1)
A(a2)
C(c1)
C(c2)
N(c2, a1)
N(a1, a2)
N(a2, c1)
N(c2, e(a1))

$\forall x,y\ [N(x,y) \rightarrow N(y,x)]$

$\forall x,y,z\ [(N(x,y) \wedge N(y,z) \wedge x \neq z) \rightarrow E(x,y,z)]$



N(c2, a1) é verdade se N = "ao lado"
N(c2, a2) é falso se N = "ao lado"

Formalizar a Teoria:

conceptualização (descrever um mundo) (D, F, R)

Entidades do Mundo (universo)
ex: Sherlock, ♀♀, anita

Conjunto de funções (funções de aridade n tuplo de aridade n+1 de entidades de D)
ex: {(△, ♀), (♀, ♀), (♀, △)}
input → output

Conjunto de Relações (predicados) podem ser aplicados aos elementos de D
ex: {{△}, {△}}
(vai estar associado ao predicado casas)

Exemplo:

Universo:
D = {♀, ♀, △, △}

Função:
I(c) f_1 = {(♀, △), (♀, ♀), (△, ♀)}
I(c) f_2 = {(△, ♀), (♀, ♀), (♀, △)}
F = {f_1, f_2}

Relações:
I(A) = R_1 {(♀), (♀)}
I(C) R_2 = {(△)(△)}
I(N) R_3 = {(△, ♀), (♀, △), (♀, ♀), (♀, ♀), (♀, △), (△
I(E) R_4 = {(△, ♀, ♀), (♀, ♀, △)}
R = {R_1, R_2, R_3, R_4}

INTERPRETAÇÃO: Para (D, F, R)
1. cada constante f_i° é associada a uma entidade de D. ex: a1, a2, c1, c2 ← constantes
2. cada função f é associada a uma função de F com aridade n;
3. Cada letra de predicado P, é associada a uma relação R com aridade n.

CONDIÇÕES DE SATISFAÇÃO:
I satisfaz α se α = (P(t1, ..., tn))
I satisfaz α sse I(t1), ..., I(tn) ∈ I(P) => I satisfaz ∀x [α] se I satisfaz α ∈ {a/x} para qualquer substituição
I satisfaz ¬α se I não satisfaz α

ex: A(a1) ? → A(a1) = P(t1)
I(a1) = (♀) → ♀ pertence a I(A) = {(♀, ♀)}, logo satisfaz
N(z2, e(a2))?
↳ (I(c2), I(e)(I(a2))
(△, I(e)(♀) → (△, ♀) pertence a I(N) logo satisfaz

Resumindo:

LÓGICA PROPOSICIONAL
sistema de dedução natural → LÓGICA de 1ª ORDEM sistema de dedução natural → Programação em Lógica → Prolog
sistema semântico (tabelas, OBDD's, SAT) → sistema semântico

LÓGICA DE 1ª ORDEM: fbf α → Verdadeira segundo uma interpretação I
satisfazível
contraditória
tautológica
falsificável

Modelo: interpretação que satisfaz todas as fbf de um conjunto
↳ se existe para um conjunto de fbf => satisfazíveis

consequência semântica Δ ⊨ α se todos os modelos de Δ são modelos de α

Teorema da Refutação Δ ⊨ α sse Δ ∪ {¬α} é contraditória

LPO **correta** se Δ ⊢ α => Δ ⊨ α, e **completa** se Δ ⊨ α => Δ ⊢ α

Mania Ribeiro
11/06/2019